WeMakeDevs x FalkorDB Graph Hackathon | Track 01: Agents That Act on Connected Data

Blast Radius

A graph-grounded agent that tells you whether a vulnerable dependency is actually reachable from your code, shows the exact path, and remembers what it found.

Author	Utkarsh Sonthalia
Date	12 October 2026
Status	Working prototype (embedded FalkorDB, Claude agent, seeded sample codebase)
Primary track	01 - Agents That Act on Connected Data
Secondary fit	02 - Agent Memory (findings persist in the graph across sessions)

Executive summary

Dependency scanners report every vulnerable package in a project, but they do not say whether the vulnerable function is ever called, from which entry point, or who owns the code in between. Teams therefore triage long lists by hand. That question is a graph question: it is a path from a public route, through internal calls, to a package function that a vulnerability affects. Blast Radius stores the codebase as a FalkorDB graph and gives a Claude agent graph tools to answer it. Every answer carries the path it came from, and every conclusion is written back to the graph so later sessions start from what is already known.

1. Problem

The hackathon brief describes agents that see pieces instead of the whole picture, lose track of earlier sessions, and cannot explain their choices. Vulnerability triage shows all three:

- **Pieces, not the picture.** A scanner lists a vulnerable package; the call graph, route table and ownership file live elsewhere. Reachability needs all of them joined.
- **No memory.** The same advisory is re-investigated by different people every release because the conclusion ("only reachable from an internal script") is never recorded next to the code.
- **No explanation.** An LLM asked "are we affected?" over raw files will guess. A reviewer needs the concrete chain of calls before acting.

Reachability is multi-hop by nature: route → handler → zero or more internal calls → package function ← vulnerability. A variable-length path query expresses this in one line; in a relational store it is a recursive join, and in a vector store it is not expressible.

2. Why a graph, and why FalkorDB

Need	How the graph meets it
Follow connections across data	Routes, functions, modules, packages, advisories and people are nodes; calls and ownership are edges. One Cypher pattern spans all of them.
Multi-hop reasoning	Variable-length match (CALLS*0..6) finds every chain from any route to the vulnerable function.
Show the path behind an answer	The query returns nodes(path), which the agent quotes verbatim as evidence.
Remember across sessions	Findings are nodes linked by ABOUT edges to the entities they concern, so recall is a graph lookup, not a text search.
Connect people and decisions	OWNS edges turn a technical path into an assignee.

FalkorDB is the required context layer for the hackathon. It speaks openCypher and has a Python client. For local testing the prototype uses *falkordblite*, an embedded build that needs no Docker or server. Setting FALKORDB_HOST switches the same code to a FalkorDB server or FalkorDB Cloud without other changes.

3. Architecture

Layer	Component	Responsibility
Data	seed.py (later: ingest from repo)	Builds the code graph: routes, functions, calls, modules, owners, packages, advisories.
Context	FalkorDB graph 'codegraph'	Single source of truth for structure and for agent memory (Finding nodes).
Tools	tools.py	graph_schema, run_cypher (read-only), exposure_paths, owners_of, remember, recall. Every call is logged to a trace.
Agent	agent.py (Claude Opus 5.5, tool runner)	Plans which tools to call, answers only from tool results, writes conclusions back.
Output	Console (later: web UI)	Answer + the graph path(s) + owner + the ordered tool trace.

3.1 Graph schema

```
(Route)-[:HANDLED_BY]->(Function)
(Function)-[:CALLS]->(Function)
(Function)-[:CALLS]->(PackageFunction)
```

```
(Function)-[:DEFINED_IN]->(Module)<-[:OWNS]-(Person)
(Package)-[:EXPORTS]->(PackageFunction)<-[:AFFECTS]-(Vulnerability)<-[:HAS_VULN]-(Package)
(Finding)-[:ABOUT]->(any node)            // agent memory
```

3.2 The core query

```
MATCH (v:Vulnerability {id: $id})-[:AFFECTS]->(pf:PackageFunction)
MATCH path = (r:Route)-[:HANDLED_BY]->(:Function)
             -[:CALLS*0..6]->(:Function)-[:CALLS]->(pf)
RETURN r.method + ' ' + r.path AS route, r.public AS public,
       [n IN nodes(path) | coalesce(n.name, n.path)] AS chain
```

A companion query lists every direct caller of the vulnerable function and how many routes reach it. Callers with zero routes (for example a migration script) are reported as present but not exposed, which is the distinction scanners miss.

3.3 Agent loop and safety

- Claude Opus 5.5 through the Anthropic Python SDK tool runner, with adaptive thinking and effort set to medium.
- The system prompt requires answers to come only from tool output, and asks the agent to call recall first and remember last.
- run_cypher rejects write keywords and executes through FalkorDB's read-only query path, so the model cannot alter the graph except through the narrow remember tool.
- Server-side refusal fallback is enabled, and a refusal stop reason is handled explicitly.

4. Prototype results

The sample codebase is a small Node-style service: 3 routes, 9 functions, 7 modules, 3 owners, 4 packages and 3 advisories. All advisory IDs are fictional (DEMO-*) test data. The results below are the actual output of `demo_offline.py`, which runs the graph tools without the LLM.

Figure 1. The exposure path the agent returns as evidence (real output from the prototype).

Advisory	Affected function	Exposed routes (chain)	Not exposed	Owner to act
DEMO-2026-0001 (critical)	lodash _.merge	PUBLIC POST /orders: createOrder -> applyDiscounts -> _.merge	deepMergeConfig (scripts only, 0 routes)	bob (src/lib/merge.js)
DEMO-2026-0002 (high)	jsonwebtoken jwt.verify	PUBLIC POST /orders and internal GET /reports/:id via verifySession	-	alice (src/lib/tokens.js)
DEMO-2026-0003 (high)	ejs ejs.render	internal GET /reports/:id: exportReport -> renderTemplate	-	carol (src/lib/render.js)

What this demonstrates: the same critical advisory has two callers, but only one is reachable from a public entry point. A flat scanner would rate both equally. The graph separates "fix today" from "fix when convenient" and names the owner. Memory was verified by storing a finding about DEMO-2026-0001 and recalling it by advisory ID. The write guard was verified by attempting `DETACH DELETE`, which was rejected.

Not yet verified: the full Claude-driven run (agent.py) needs an Anthropic API key, which was not configured on the build machine. Tool schemas were generated and validated locally.

5. Mapping to the judging focus

Track 01 key focus	Where it shows up
Multi-hop reasoning	Variable-length CALLS path from route to advisory
Graph queries and algorithms as agent tools	exposure_paths, owners_of, run_cypher, graph_schema
Tool selection through MCP	Planned: expose tools.py as an MCP server (same functions, new transport)
Actions based on what the graph shows	Planned: open an issue or PR assigned to the owner on the path
Explainable results	Every answer includes the node chain and the ordered tool trace

6. Evaluation plan

- **Ground truth.** Seed graphs with known reachable and unreachable advisories (including decoys such as dead code and test-only callers).
- **Metrics.** Precision and recall on "is this advisory reachable from a public route"; path correctness (exact chain match); owner correctness; number of tool calls per answer.
- **Baselines.** (a) scanner output alone, where every vulnerable package counts as affected; (b) the same LLM with raw source files and no graph.

- **Memory test.** Ask the same question in a second session; measure whether the agent reuses the stored finding and how many fewer tool calls it needs.

7. Roadmap

Step	Work	Outcome
1	Real ingestion: parse a JS/TS or Python repo (imports, call edges via tree-sitter), CODEOWNERS/git blame for owners	Graph built from a real public repo
2	Advisory feed: map OSV advisories to package functions	Real, not seeded, vulnerabilities
3	MCP server wrapping tools.py	Satisfies the MCP tool-selection focus
4	Action tool: draft a GitHub issue with the path and the owner	Agent acts, not just reports
5	Web UI that highlights the returned path in a graph view	Clear demo video
6	Run the evaluation suite and report numbers in the submission	Evidence for judges

8. Risks and limits

- Static call graphs miss dynamic dispatch, reflection and callbacks. Mitigation: mark edges with confidence and report possible paths separately.
- Mapping an advisory to the exact affected function is often manual. Mitigation: fall back to package-level reachability and label it as such.
- Embedded FalkorDB suits local testing; the demo should run against a FalkorDB server or FalkorDB Cloud.
- The agent must not overstate safety: "no path found" means no path in the modelled graph, and the answer should say so.

9. Running the prototype

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python demo_offline.py          # graph tools only, no API key
.venv/bin/python seed.py                  # reset the sample graph
export ANTHROPIC_API_KEY=...
.venv/bin/python agent.py "Which public routes are exposed to DEMO-2026-0001, and who should fix it?"
```